

SMARTMOVE TRANSPORT SOLUTIONS

MEMBER 1 - FULL RESPONSIBILITIES, JOBS & WORK PLAN

System Analyst + ER/EER Designer + Database Architect + Requirements Owner

Purpose of this document

This document is the complete internal handoff for Member 1. It explains exactly what Member 1 owns, what must be designed, what evidence must be produced, what must be handed to the coding members, and how Member 1 supports the final presentation and viva.

Project fact	What the CW sheet says
Module	Data Management 2
Assessment	CW No. 1 - Group mode
Maximum group size	4 members
Assessment type	Software Presentation VIVA
Submission detail	GitHub link of the project
Main deliverables	Presentation + Software Application with Database Implementation
Core database	Oracle Database
Complementary NoSQL	MongoDB
PL/SQL requirement	Procedures, functions, cursors and/or triggers with exception handling
Business reports	At least 5

Source basis: NIBM Data Management 2 CW No. 1 assessment sheet supplied in this conversation. Recommendations in this document are team-planning guidance built around the stated rubric; they are not additional lecturer requirements.

Academic integrity warning

The CW sheet states that AI generated contents will obtain 0 marks. Use this document as an internal planning and understanding guide. The team should verify, implement, test and explain its own work and follow the lecturer policy.

1. Member 1 Role at a Glance

Member 1 is the system-analysis and database-design owner. The main job is to convert the CW scenario into a precise, agreed system model that the technical team can implement without guessing.

Area	Member 1 ownership	Final result
Requirements	Extract every required feature and rule from the CW	Requirements checklist
System analysis	Define users, use cases, workflows and business rules	System specification
ER/EER	Design the complete relational model	Approved ERD/EERD
Normalization	Validate 1NF/2NF/3NF and resolve redundancy	Normalization notes
Oracle design	Define entities, attributes, keys, constraints and relationships	Oracle schema blueprint
Data dictionary	Define what every important field means	Data dictionary
Data integrity	Define integrity rules before implementation	Constraint specification
Sample data plan	Specify realistic data coverage needed for testing/reporting	Seed-data plan
Cross-team handoff	Give Members 2-4 a frozen design baseline	Implementation handoff pack
QA support	Check that implementation still matches the approved design	Design QA checklist
Presentation/viva	Explain requirements, ERD, normalization and design decisions	Presentation section + viva answers

The single most important responsibility

Make the database and system design correct before heavy coding starts. Member 1 should prevent the team from building contradictory tables, duplicated data, missing relationships, or features that do not satisfy the CW.

REQUIREMENTS -> SYSTEM MODEL -> ER/EER -> NORMALIZATION -> DATA DICTIONARY -> ORACLE BLUEPRINT -> CODING HANDOFF

2. What Member 1 Must Cover from the CW

The assessment sheet requires a transportation system for SmartMove Transport Solutions. The database must support nine core operations, and the complete project also includes PL/SQL, five business reports, MongoDB integration, a web/enterprise application, presentation and viva.

Nine core operations

- Vehicle Management
- Driver Management
- Route Management
- Passenger Management
- Trip Scheduling Management
- Ticket Booking Management
- Payment Processing
- Maintenance Management
- Feedback and Review Management

Member 1 must design for these, even when another member implements them

Feature	What Member 1 must define
Vehicle	Vehicle identity, registration, type, capacity, status and relationships
Driver	Driver identity, licensing/qualification details and assignment relationships

Route	Route identity, origin, destination and any route attributes required by the system
Passenger	Passenger identity and contact/account information needed by the system
Trip	Trip instance, schedule, route, vehicle and driver relationships
Booking	Passenger-to-trip booking relationship, booking status and seat/ticket information
Payment	Payment record tied to a booking, amount/status/date and integrity rules
Maintenance	Vehicle maintenance history, maintenance dates/status and due logic
Feedback	Passenger feedback relationship to the relevant service context

What the CW does NOT explicitly require from Member 1

Important boundary

The CW does not explicitly say that Member 1 must write the presentation, write all PL/SQL, code the website, or build MongoDB. Member 1 designs and specifies the parts that make those implementations coherent. Ownership is about accountability, not doing every technical task personally.

3. Workstream A - Build the Master Requirements Specification

Do this first. Every later design decision must trace back to a requirement or an agreed supporting decision.

Step A1 - Extract the mandatory requirements

- Design an ER or EER diagram.
- Develop a simple Web or Enterprise Application.
- Use Oracle Database for the relational database schema.
- Support the nine core operations listed in the CW.
- Populate realistic sample data.
- Implement PL/SQL business logic using procedures, functions, cursors and/or triggers with exception handling.
- Provide at least five business reports.
- Use MongoDB as complementary NoSQL storage for dynamic/unstructured transportation information.
- Capture vehicle images/documents, passenger reviews/feedback, announcements/notifications and trip multimedia in MongoDB.
- Demonstrate the four required MongoDB query scenarios.
- Integrate MongoDB with the application layer.
- Prepare the presentation and software/database implementation for submission.

Step A2 - Convert requirements into acceptance statements

Write each requirement as something the team can test. Example:

Requirement: Ticket Booking Management

Acceptance test: A passenger can select a scheduled trip, create a booking, and the system stores a valid booking linked to the passenger and trip.

Step A3 - Create the master traceability matrix

ID	Requirement	Database object(s)	API/UI owner	Evidence
REQ-01	Vehicle Management	VEHICLE	M3 + M4	CRUD + DB screenshot
REQ-02	Driver Management	DRIVER	M3 + M4	CRUD + DB screenshot
REQ-03	Route Management	ROUTE	M3 + M4	CRUD + DB screenshot
REQ-04	Passenger Management	PASSENGER	M3 + M4	CRUD + DB screenshot
REQ-05	Trip Scheduling	TRIP + relationships	M3 + M4	Schedule demo
REQ-06	Ticket Booking	BOOKING	M3 + M4	Booking demo
REQ-07	Payment Processing	PAYMENT	M3 + M4	Payment demo
REQ-08	Maintenance	MAINTENANCE	M3 + M4	Maintenance demo/report
REQ-09	Feedback/Review	FEEDBACK + Mongo REVIEWS	M3 + M4	Review demo
REQ-10	Five reports	Oracle report logic	M2 + M3 + M4	Report outputs

Member 1 must keep this matrix updated

Whenever a table, relationship or feature changes, update the requirement mapping. This becomes the teams completeness check before submission.

4. Workstream B - Identify Users, Actors and Responsibilities

Before the ERD, define who uses the system and what they are allowed to do. This prevents missing entities and unclear workflows.

Actor	Main responsibilities in SmartMove	Relevant system areas
Administrator	Manage master data, operations, maintenance and reports	Vehicles, drivers, routes, trips, maintenance, reports

Driver	View/handle assigned trip information	Driver profile, trip assignment
Passenger	Register, search trips, book tickets, pay and submit feedback	Passenger, trip, booking, payment, feedback

Create use-case inventory

- Register passenger
- Manage vehicles
- Manage drivers
- Manage routes
- Schedule trip
- Book ticket
- Process payment
- Record maintenance
- Submit feedback
- Generate reports
- View announcements
- View vehicle documents/media

Business rules to identify before the ERD

- A booking must reference a valid passenger and valid trip.
- A payment must be traceable to the relevant booking.
- A trip should reference the route it operates and the vehicle/driver arrangement used by the system.
- A maintenance record must reference a valid vehicle.
- Report calculations must be based on consistent transactional data.
- The system must prevent impossible or duplicate relationships where business rules require uniqueness.
- MongoDB content should have a clear reference back to the relevant relational entity when cross-system linkage is needed.

5. Workstream C - Design the Complete ER/EER Diagram

This is the most visible technical artifact owned by Member 1. The rubric expects the ER diagram to be complete, normalized and clearly represent all entities for the highest band.

Step C1 - Draft the entity list

Entity	Why it exists	Likely key
PASSENGER	Stores passenger master data	passenger_id
DRIVER	Stores driver master data	driver_id
VEHICLE	Stores vehicle master data	vehicle_id
ROUTE	Stores route definitions	route_id
TRIP	Represents scheduled trip instances	trip_id
BOOKING	Represents a passenger reservation/ticket	booking_id
PAYMENT	Represents payment transactions	payment_id
MAINTENANCE	Stores vehicle maintenance history	maintenance_id
FEEDBACK	Stores structured relational feedback if used	feedback_id

Step C2 - Resolve relationship cardinality

Relationship	Expected thinking	Design question
Passenger -> Booking	One passenger can have many bookings	Is BOOKING the child/transaction entity?
Trip -> Booking	One trip can have many bookings	How will booked capacity be derived?
Route -> Trip	A route can be used by many trip instances	Should route details be duplicated? No.
Vehicle -> Maintenance	One vehicle can have many maintenance	How is maintenance history preserved?

	records	
Vehicle/Driver -> Trip	Trip needs operational assignment data	Do we store direct FKs or an assignment entity?
Booking -> Payment	Payment must be traceable to booking	Can one booking have more than one payment record? Decide and document.

Step C3 - Decide whether an associative entity is necessary

If a relationship is many-to-many, do not hide it. Resolve it using an associative/bridge entity. Example: if a driver can be assigned to many trips and a trip could involve assignment history or multiple drivers, document the exact model chosen.

Step C4 - Validate the ERD

- Every required CW operation has at least one supporting entity.
- Every entity has a clear primary key.
- Foreign keys represent actual relationships, not guessed links.
- There are no unnecessary duplicated attributes across parent/child entities.
- Cardinalities are shown consistently.
- Relationship names make business meaning clear.
- The model supports the five reports.
- The model supports realistic sample data.
- The model leaves room for the application to reference MongoDB content without duplicating dynamic data into Oracle unnecessarily.

6. Workstream D - Normalization and Data Quality

Member 1 must be able to explain why the proposed relational design avoids common update, insert and delete anomalies.

1NF

Each field should represent an atomic value rather than a repeating group or comma-separated list.

2NF

After 1NF, non-key attributes should depend on the whole key. This matters especially when a table uses a composite key.

3NF

After 2NF, non-key attributes should not depend on another non-key attribute. Separate entities when transitive dependency creates redundancy.

Normalization review questions

- Could the same passenger phone number appear in hundreds of booking rows?
- Could route origin/destination be repeated in every trip row?
- Could vehicle capacity be copied into every booking?
- Could changing a driver detail require editing many trip records?
- Could a maintenance update accidentally overwrite historical maintenance data?

Deliverable: normalization notes

Create a short internal sheet showing at least 2-3 representative examples of how the design moved from a problematic structure to a normalized structure. Keep it clear enough for the viva.

7. Workstream E - Build the Data Dictionary

The data dictionary is the bridge between Member 1s design and Member 3s SQL implementation.

Table	Column	Type concept	Key/Rule	Meaning
PASSENGER	passenger_id	Numeric identifier	PK, NOT NULL	Unique passenger
PASSENGER	name	Text	NOT NULL	Passenger display name
VEHICLE	vehicle_id	Numeric identifier	PK	Unique vehicle
TRIP	route_id	Numeric identifier	FK	Route used for the trip
BOOKING	passenger_id	Numeric identifier	FK	Passenger making the booking
BOOKING	trip_id	Numeric identifier	FK	Trip being booked
PAYMENT	booking_id	Numeric identifier	FK/unique depending design	Booking being paid for
MAINTENANCE	vehicle_id	Numeric identifier	FK	Vehicle under maintenance

For every important field, decide

- Data type concept
- Nullable or mandatory
- Primary/foreign/unique key
- Business meaning
- Allowed values or range
- Whether an index may be useful
- Whether it appears in reports or UI

8. Workstream F - Produce the Oracle Database Blueprint

Member 1 designs the Oracle structure; Member 3 is responsible for implementing it. Do not let both members independently invent the schema.

Oracle object plan

Layer	Member 1 decides	Member 3 implements
Tables	Entity and column design	CREATE TABLE
Keys	PK/FK strategy	PRIMARY KEY / FOREIGN KEY
Constraints	Mandatory values and valid states	NOT NULL / CHECK / UNIQUE
Sequences/identity	ID generation strategy	Sequence or supported identity approach
Indexes	Fields likely to benefit	CREATE INDEX
Views	Useful read-only/reporting abstractions	CREATE VIEW
Security	Data-access principles and role needs	Users/roles/grants as agreed
Backup	What must be protected/recovered	Backup procedure/documentation

Suggested entity-to-operation coverage

Operation	Primary Oracle entities
Vehicle Management	VEHICLE
Driver Management	DRIVER
Route Management	ROUTE
Passenger Management	PASSENGER
Trip Scheduling	TRIP + ROUTE + VEHICLE + DRIVER/assignment
Ticket Booking	BOOKING + PASSENGER + TRIP
Payment Processing	PAYMENT + BOOKING
Maintenance Management	MAINTENANCE + VEHICLE
Feedback/Review	FEEDBACK plus MongoDB REVIEW content where appropriate

Design rule

Never add a table just because it sounds impressive. Every table must have a business purpose, a relationship, and a reason it is stored in the relational model.

9. Workstream G - Define Data Integrity Rules

Member 1 owns the decisions; Member 3 converts them into Oracle constraints and backend validation.

Rule category	Example decision for SmartMove
Entity integrity	Every passenger, vehicle, driver, route, trip, booking and payment has a unique ID.
Referential integrity	Booking cannot reference a nonexistent passenger or trip.
Domain integrity	Rating/status/amount/date fields must stay within valid ranges or defined states.
Uniqueness	Business identifiers such as registration number may need uniqueness.
Mandatory fields	Critical fields required for a usable record cannot be NULL.
State consistency	Status values are controlled and consistent across the system.

Checklist before handoff

- ☐ Every foreign key has a parent entity.
- ☐ Every status field has a defined allowed set.
- ☐ Amount/date fields have sensible validation rules.
- ☐ Duplicate business identifiers are prevented where required.
- ☐ Historical records are preserved instead of overwritten when history matters.

- [] Report calculations have reliable source columns.

10. Workstream H - Design the Sample Data Plan

The CW requires sufficient sample data to simulate a realistic application scenario. Member 1 does not need to type all data personally, but must make sure the data design is rich enough to test the model and reports.

Data set	Recommended purpose	Coverage to ensure
Passengers	Bookings/history	Multiple passengers with different booking histories
Drivers	Trips/performance	Several drivers with repeated trip assignments
Vehicles	Trips/maintenance	Different vehicle types/statuses and some maintenance due
Routes	Usage/revenue reports	Several routes with different booking frequencies
Trips	Scheduling/occupancy	Past, current and future scheduled trips if supported
Bookings	Revenue/history	Multiple bookings distributed across routes/trips
Payments	Revenue	Successful/other relevant statuses according to business rules
Maintenance	Maintenance report	Records with different next-due dates/statuses
Feedback	Reviews/ratings	Positive and negative ratings/comments for reporting/search

Member 1 must coordinate report-friendly data

A report cannot demonstrate anything useful when there is only one route, one passenger or three bookings. Check the sample-data plan against each report before the team starts demos.

Report	Data dependency to verify
Most used routes	Many bookings across multiple routes
Revenue by period	Payments across multiple dates and/or trips
Passenger history	At least one passenger with multiple bookings/trips
Vehicles due maintenance	Maintenance records with different due dates/statuses
Trip occupancy/performance	Trips with different capacity and booking counts

11. Workstream I - Member 1 Handoff to the Other Three Members

Member 1 should formally hand over a stable design pack instead of sending random screenshots or changing the ERD without notice.

To Member 2 - Theory/Reporting Member

- Final entity list and relationship list.
- Approved ERD/EERD.
- Normalization reasoning.
- Data dictionary.
- Business rules needed for PL/SQL specification.
- Data dependencies for the five reports.
- Definitions of what each database object means for viva explanations.

To Member 3 - Backend/Oracle Developer

- Frozen Oracle schema blueprint.
- Table/column/key definitions.
- Constraint requirements.
- Relationship/cardinality decisions.
- Sample-data coverage requirements.
- Report data dependencies.
- Business rules that must be enforced in Oracle/backend.
- Approved changes only - no silent schema drift.

To Member 4 - Frontend/MongoDB Developer

- Entity names and field meanings for UI labels/forms.
- Status/value definitions.
- Relationships needed for displaying linked data.
- Which relational IDs should be displayed/referenced in the UI.
- Which dynamic content belongs in MongoDB rather than duplicated in Oracle.

Handoff format

Store the approved design in GitHub so there is one authoritative version. Recommended folders: docs/requirements, docs/erd, docs/database-design, docs/data-dictionary.

12. Workstream J - Change Control and Design QA

A common group-project failure is schema drift: one person changes a column, another still uses the old name, and the frontend breaks. Member 1 is the first gatekeeper against this problem.

Rule for every proposed schema change

1. State what is changing.
2. State why it is needed.
3. Identify which requirement it supports.
4. Check impact on ERD.
5. Check impact on PL/SQL.
6. Check impact on API/backend.
7. Check impact on frontend.
8. Check impact on reports.
9. Update the data dictionary.

10. Then merge the change.

Design QA questions

- Does every requested CW feature map to a database structure?
- Can every report be calculated from the Oracle model?
- Can every important transaction be traced through foreign keys?
- Are dynamic MongoDB records able to reference relational IDs where useful?
- Is any important business data unnecessarily duplicated?
- Will the sample data let the lecturer see meaningful results?

13. Workstream K - Evidence and Proof of Completion

Even though the formal submission listed on the sheet is a GitHub link plus presentation/software, the team should maintain internal evidence so the work can be demonstrated confidently.

Evidence item	Member 1 action
ERD	Keep final export and source/editable file
Normalization	Keep concise before/after reasoning
Data dictionary	Keep final version aligned with schema
Requirement matrix	Mark implementation status as complete
Schema review	Keep screenshots or commit references for major milestones
Sample-data plan	Keep evidence that report scenarios have enough data
Design QA	Record issues found and resolution

Suggested GitHub structure for Member 1

```
docs/
  requirements/
    requirements-matrix.md
    use-cases.md
    business-rules.md
  erd/
    SmartMove-ERD.png
    SmartMove-ERD.pdf
  database-design/
    schema-blueprint.md
    constraints.md
    normalization.md
  data-dictionary/
    data-dictionary.xlsx-or-csv
  qa/
    design-review-checklist.md
```

14. Workstream L - Presentation and Viva Responsibilities

Member 1 should be able to present the design with confidence, not simply show the diagram.

Member 1 presentation section

- Problem and system context
- Requirements summary
- Actors/use cases
- System data model
- ER/EER diagram
- Normalization rationale
- Key business rules
- How the Oracle design supports the required operations

Viva questions Member 1 must master

- Why did you separate Passenger, Booking and Trip?
- What is the difference between a Route and a Trip in your model?
- Why is Payment separate from Booking?
- How is Vehicle related to Maintenance?
- What is the cardinality between Passenger and Booking?
- Where are foreign keys used and why?
- What normalization problems did you avoid?
- Why is your model in 3NF?
- Why should route information not be copied into every booking row?
- How does your schema support the revenue report?
- How does your schema support passenger travel history?
- How does your schema support the maintenance report?
- What data belongs in Oracle versus MongoDB?
- How can MongoDB content link back to Oracle data?
- What happens when an invalid foreign key is used?
- What would you change if the company later adds another vehicle type or service?

15. Workstream M - Support Oracle + MongoDB Integration Design

Member 1 is not the MongoDB coder, but should make sure the relational model and the MongoDB model can coexist cleanly.

Oracle structured data	MongoDB dynamic data	Linking idea
VEHICLE	vehicle_documents / images	vehicleId
PASSENGER	review content	passengerId
ROUTE / TRIP	reviews / trip media / announcements	routeId / tripId
DRIVER	feedback/rating content	driverId

Design principle

Store core transactional facts in Oracle and dynamic/unstructured content in MongoDB, while keeping a clear identifier relationship when cross-system display or analytics needs it. The CW explicitly describes MongoDB as complementary to the relational database.

16. Member 1 Complete Checklist

Do not mark the role complete until these are checked.

Requirements and analysis

- ☐ Read the CW requirements fully and extracted mandatory items.
- ☐ Created the master requirements matrix.
- ☐ Identified system actors and use cases.
- ☐ Documented major business rules.
- ☐ Mapped each requirement to design/data/evidence.

ER/EER and normalization

- ☐ Complete ER/EER created.
- ☐ PKs defined.
- ☐ FKs defined.
- ☐ Cardinalities/relationships validated.
- ☐ Many-to-many cases resolved.

- ☐ 1NF/2NF/3NF reasoning prepared.
- ☐ ERD supports all nine core operations.
- ☐ ERD supports the five report data needs.

Oracle design

- ☐ Table blueprint complete.
- ☐ Data types/concepts documented.
- ☐ NOT NULL/UNIQUE/CHECK requirements documented.
- ☐ Index candidates identified.
- ☐ Sequence/ID strategy agreed.
- ☐ Security/privilege needs identified with Member 3.
- ☐ Backup/recovery expectations discussed with Member 3.

Data quality and testing

- ☐ Realistic sample-data plan prepared.
- ☐ Each report has enough sample data.
- ☐ Design QA completed after implementation.
- ☐ Schema changes tracked and approved.

Handoff and viva

- ☐ Member 2 received design pack.
- ☐ Member 3 received implementation blueprint.
- ☐ Member 4 received UI data definitions.
- ☐ Final ERD matches implemented schema.
- ☐ Presentation section prepared.
- ☐ Viva questions understood without memorized nonsense.

17. Definition of Done - Member 1

Member 1 is finished only when the design is implementation-ready.

Implementation-ready means another developer can create the Oracle schema and application features from the design pack without guessing what an entity, attribute, key, relationship or business rule means.

Level	Done means
Minimum	All mandatory entities and relationships are identified.
Good	ERD, normalization, data dictionary and requirements matrix are aligned.
Strong	The schema supports all nine operations and all five report data needs.
Excellent	The design is traceable, normalized, realistic, implementation-ready, and confidently explainable in the viva.

18. Recommended Work Schedule for Member 1

Stage	Primary work	Exit condition
Day 1	Requirements extraction	Every CW requirement mapped
Day 2	Actors, use cases, business rules	Core workflows documented
Day 3	Draft ERD	Initial entity/relationship model complete
Day 4	Normalization + data dictionary	Model can be justified
Day 5	Oracle schema blueprint	M3 can implement without guessing
Day 6	Sample-data and report dependency review	Data can support all reports
Ongoing	Design QA with M3/M4	Implementation stays aligned
Final phase	Presentation/viva preparation	Can explain every major design decision

19. Member 1 Rules - What to Do and What to Avoid

Do	Avoid
Freeze the design before heavy coding.	Changing table names casually after APIs are built.
Explain why each entity exists.	Adding tables only to make the ERD look large.
Use consistent naming.	PassengerID in one place and passenger_id elsewhere.
Trace every requirement.	Assuming “the app probably covers it.”
Coordinate with M2/M3/M4.	Working in isolation.
Validate report data needs.	Discovering at the end that revenue cannot be calculated.
Document changes.	Silent schema drift.
Know the model deeply.	Memorizing an ERD without understanding relationships.

20. Final Handoff Pack You Must Give the Team

File / artifact	Purpose	Receiver
requirements-matrix.md	Complete CW coverage	M2 + M3 + M4
SmartMove-ERD.png/pdf	Authoritative relational model	All members
normalization.md	Explain data-quality design	All members
data-dictionary	Field-level implementation reference	M3 + M4
schema-blueprint.md	Oracle implementation contract	M3
constraints.md	Integrity rules	M3
business-rules.md	Rules for PL/SQL/backend/UI	M2 + M3 + M4
sample-data-plan.md	Ensure realistic report/test data	M3 + M4
design-review-checklist.md	Validate final implementation	All members

Final mindset

Your role is not “the person who draws the ERD.” Your role is to make the SmartMove system understandable, traceable and structurally correct before and during implementation. When Member 3 codes and Member 4 builds the UI, they should be implementing YOUR agreed model, not inventing a second one.

Appendix - Source and Scope Note

This work package is based on the NIBM Data Management 2 CW No. 1 assessment sheet supplied by the user. The source sheet states the SmartMove Transport Solutions scenario, the nine core operations, Oracle and PL/SQL requirements, minimum five business reports, MongoDB requirements and query demonstrations, deliverables, and marking rubric.

Where this document gives exact counts, folder structures, workflow order, sample-data scale, or recommended technologies, those are planning recommendations for the team rather than quoted lecturer requirements.